

Inteligência Artificial Local com Ollama

Um guia prático do zero ao servidor de IA

Paulo Pimenta

2026-05-02

Contents

1	Introducao	3
2	O Ollama e os Modelos de IA	3
2.1	O que e o Ollama?	3
2.2	O que sao Modelos de Linguagem (LLMs)?	3
2.3	Principais modelos disponiveis no Ollama	3
2.4	Tipos de modelos por finalidade	4
2.4.1	Modelos generalistas	4
2.4.2	Modelos especialistas em codigo	4
3	Hardware e Configuracao dos Modelos	4
3.1	Por que o hardware importa?	4
3.2	Componentes criticos	4
3.2.1	GPU (Placa de Video)	4
3.2.2	RAM	5
3.2.3	CPU	5
3.3	Tabela de recomendacoes por hardware	5
3.4	Exemplos praticos de configuracao	5
3.4.1	Notebook com CPU antiga e GPU integrada	5
3.4.2	Notebook moderno com GPU dedicada de entrada	5
4	Instalando o Ollama	6
4.1	Instalacao no Windows	6
4.1.1	Requisitos	6
4.1.2	Passo a passo	6
4.2	Instalacao no Linux (Ubuntu 22.04 LTS)	6
4.2.1	Requisitos	6
4.2.2	Passo a passo	6

5	Instalando Modelos pelo Ollama	7
5.1	O comando ollama pull	7
5.2	Exemplos praticos	7
5.2.1	Windows (PowerShell)	7
5.2.2	Linux (Terminal)	7
5.3	Gerenciando modelos instalados	8
5.4	Monitorando o espaco em disco	8
5.4.1	Windows	8
5.4.2	Linux	8
6	Usando os Modelos	8
6.1	Uso via linha de comando	8
6.2	Uso via Python	9
6.2.1	Instalacao da biblioteca	9
6.2.2	Exemplo basico	9
6.2.3	Chat com historico de conversa	9
6.2.4	Streaming de respostas	9
6.2.5	Exemplo aplicado a ciencia de dados	10
6.3	Uso via API REST	10
7	Integrando Modelos nas IDEs com o Plugin Continue	10
7.1	O que e o Continue?	10
7.2	Instalacao no VS Code	11
7.3	Configuracao do arquivo config.yaml	11
7.3.1	Configuracao com um unico modelo	11
7.3.2	Configuracao com multiplos modelos	11
7.4	Explicacao dos campos do config.yaml	12
7.5	Principais atalhos do Continue no VS Code	12
7.6	Referenciando contexto no chat	12
8	Criando um Servidor de IA com Ollama	13
8.1	Conceito	13
8.2	Configuracao no Linux (Ubuntu)	13
8.3	Configuracao no Windows	13
8.4	Acessando o servidor remotamente	14
8.4.1	Via Python	14
8.4.2	Via Continue no VS Code de outro computador	14
8.4.3	Via API REST (qualquer dispositivo)	14

8.5	Servidor em dispositivo embarcado: Jetson Nano	14
8.6	Boas praticas de seguranca para o servidor	15
9	Referencias e Recursos	15

1 Introducao

Este tutorial apresenta de forma pratica e objetiva como utilizar modelos de inteligencia artificial (IA) localmente, sem depender de servicos em nuvem. Abordaremos desde os conceitos fundamentais ate a criacao de um servidor de IA acessivel por multiplos dispositivos na mesma rede.

A execucao local de modelos de linguagem (LLMs) oferece vantagens importantes:

- **Privacidade:** os dados nao saem do seu computador
 - **Custo zero:** sem limites de uso ou assinaturas
 - **Disponibilidade offline:** funciona sem internet
 - **Controle total:** voce escolhe e gerencia os modelos
-

2 O Ollama e os Modelos de IA

2.1 O que e o Ollama?

O **Ollama** e uma plataforma de codigo aberto que permite baixar, gerenciar e executar modelos de linguagem grandes (LLMs) localmente, diretamente no seu computador. Ele abstrai toda a complexidade tecnica de executar esses modelos, oferecendo uma interface simples via linha de comando e uma API REST integrada.

Em termos praticos, o Ollama funciona como um gerenciador de modelos de IA – de modo similar ao que o `pip` e para pacotes Python ou o `apt` e para pacotes no Ubuntu.

2.2 O que sao Modelos de Linguagem (LLMs)?

Modelos de linguagem grandes (do ingles *Large Language Models*, LLMs) sao sistemas de IA treinados em enormes volumes de texto com o objetivo de compreender e gerar linguagem natural. Eles sao capazes de:

- Responder perguntas e manter conversas
- Escrever, revisar e explicar codigo
- Resumir e analisar documentos
- Auxiliar em tarefas de ciencia de dados

2.3 Principais modelos disponiveis no Ollama

A tabela a seguir apresenta os modelos mais relevantes disponiveis via Ollama, organizados por finalidade e tamanho:

Modelo	Tamanho	Especialidade	Indicado para
tinylama	637 MB	Uso geral leve	Hardware limitado
qwen2.5-coder:0.5b	400 MB	Codigo	Hardware limitado
qwen2.5-coder:1.5b	1.0 GB	Codigo e dados	Hardware intermediario
qwen2.5-coder:7b	4.7 GB	Codigo avancado	Hardware potente
llama3.2:1b	1.3 GB	Uso geral	Hardware intermediario
llama3.2:latest	2.0 GB	Uso geral	Hardware intermediario
phi3:mini	2.3 GB	Raciocinio	Hardware intermediario
deepseek-coder:6.7b	3.8 GB	Codigo avancado	Hardware potente
mistral:7b	4.1 GB	Uso geral avancado	Hardware potente

Nota: o tamanho do modelo influencia diretamente a qualidade das respostas e os recursos de hardware necessarios para executa-lo. Modelos maiores tendem a ser mais capazes, porem mais lentos em hardware limitado.

2.4 Tipos de modelos por finalidade

2.4.1 Modelos generalistas

Sao treinados para tarefas diversas como conversacao, redacao, resumo e raciocinio geral. Exemplos: llama3.2, tinylama, mistral.

2.4.2 Modelos especialistas em codigo

Treinados com grande volume de codigo-fonte em multiplas linguagens. Sao superiores aos generalistas para escrever, depurar e explicar codigo. Exemplos: qwen2.5-coder, deepseek-coder, codellama.

3 Hardware e Configuracao dos Modelos

3.1 Por que o hardware importa?

A execucao local de LLMs e intensiva em recursos computacionais. O hardware determina diretamente:

- **Quais modelos** voce consegue executar
- **A velocidade** de geracao de respostas (tokens por segundo)
- **A qualidade** maxima que voce pode alcancar

3.2 Componentes criticos

3.2.1 GPU (Placa de Video)

E o componente mais importante para executar LLMs com eficiencia. GPUs possuem centenas ou milhares de nucleos paralelos, ideais para as operacoes matriciais exigidas pelos modelos.

- **NVIDIA:** melhor suporte via CUDA – recomendada para uso com Ollama
- **AMD:** suporte via ROCm (mais limitado no Windows)
- **GPU integrada:** nao e aproveitada pelo Ollama na maioria dos casos

3.2.2 RAM

Quando o modelo não cabe inteiramente na VRAM da GPU, o excedente é carregado na RAM do sistema. Quanto mais RAM disponível, maiores os modelos que podem ser executados.

3.2.3 CPU

Utilizada quando não há GPU compatível. CPUs modernas executam LLMs de forma funcional, porém significativamente mais lenta que GPUs dedicadas.

3.3 Tabela de recomendações por hardware

Configuração	RAM	VRAM	Modelos recomendados
Hardware muito limitado	8 GB	Sem GPU dedicada	tinyllama, qwen2.5-coder:0.5b
Hardware intermediário	16 GB	2-4 GB	qwen2.5-coder:1.5b, llama3.2:1b, phi3:mini
Hardware bom	16 GB	6-8 GB	qwen2.5-coder:7b, deepseek-coder:6.7b
Hardware avançado	32 GB+	12 GB+	llama3.1:70b, deepseek-coder-v2

3.4 Exemplos práticos de configuração

3.4.1 Notebook com CPU antiga e GPU integrada

Exemplo: HP Pavilion com AMD A10-5745M (2013), 16 GB RAM, GPU Radeon HD 8610G integrada.

- Sem suporte a CUDA ou ROCm moderno
- Execução apenas na CPU – velocidade limitada (2 a 5 tokens/segundo)
- Modelos recomendados: qwen2.5-coder:0.5b e tinyllama

3.4.2 Notebook moderno com GPU dedicada de entrada

Exemplo: Dell com Intel Core i7-1165G7, 16 GB RAM, NVIDIA MX350 (2 GB VRAM).

- Suporte a CUDA – Ollama usa a GPU automaticamente
- Execução híbrida GPU + RAM para modelos até 5 GB
- Velocidade estimada: 10 a 20 tokens/segundo
- Modelos recomendados: qwen2.5-coder:1.5b, llama3.2:latest, phi3:mini

4 Instalando o Ollama

4.1 Instalacao no Windows

4.1.1 Requisitos

- Windows 10 ou superior (64 bits)
- Pelo menos 8 GB de RAM
- Espaço em disco suficiente para os modelos desejados

4.1.2 Passo a passo

1. Baixar o instalador

Acesse <https://ollama.com/download> e clique em **Download for Windows**.

2. Executar o instalador

Execute o arquivo `.exe` baixado e siga as instrucoes da tela. O Ollama sera instalado e iniciado automaticamente como um servico em segundo plano.

3. Verificar a instalacao

Abra o **PowerShell** e execute:

```
ollama --version
```

Se a instalacao foi bem-sucedida, voce vera a versao instalada, por exemplo:

```
ollama version 0.3.12
```

4. Verificar se o servico esta ativo

```
curl http://localhost:11434
```

A resposta esperada e:

```
Ollama is running
```

4.2 Instalacao no Linux (Ubuntu 22.04 LTS)

4.2.1 Requisitos

- Ubuntu 20.04 ou superior
- Pelo menos 8 GB de RAM
- Curl instalado

4.2.2 Passo a passo

1. Instalar via script oficial

```
curl -fsSL https://ollama.com/install.sh | sh
```

O script detecta automaticamente a arquitetura do sistema e instala a versao adequada.

2. Verificar a instalacao

```
ollama --version
```

3. Verificar se o servico esta ativo

```
systemctl status ollama
```

A saida deve mostrar `active (running)`.

4. Iniciar o servico manualmente (se necessario)

```
ollama serve
```

Nota: em instalacoes via script oficial, o Ollama e registrado como servico do sistema e iniciado automaticamente. Caso tenha instalado via Snap ou Flatpak, o directorio dos modelos pode estar em um caminho diferente do padrao (`~/.ollama/models`).

5 Instalando Modelos pelo Ollama

5.1 O comando ollama pull

Para baixar um modelo, utilize o comando `ollama pull` seguido do nome do modelo:

```
ollama pull <nome_do_modelo>
```

5.2 Exemplos praticos

5.2.1 Windows (PowerShell)

```
# Baixar o TinyLlama (leve, ideal para hardware limitado)  
ollama pull tinyllama
```

```
# Baixar o Qwen 2.5 Coder 0.5b (especialista em codigo, muito leve)  
ollama pull qwen2.5-coder:0.5b
```

```
# Baixar o Qwen 2.5 Coder 1.5b (especialista em codigo, intermediario)  
ollama pull qwen2.5-coder:1.5b
```

5.2.2 Linux (Terminal)

```
# Os comandos sao identicos ao Windows  
ollama pull tinyllama  
ollama pull qwen2.5-coder:0.5b
```

5.3 Gerenciando modelos instalados

```
# Listar todos os modelos instalados  
ollama list  
  
# Remover um modelo (libera espaco em disco)  
ollama rm tinyllama  
  
# Exibir informacoes detalhadas de um modelo  
ollama show qwen2.5-coder:0.5b
```

Importante: o comando `ollama rm` remove completamente os arquivos do modelo do disco. Não é necessário remover diretórios manualmente – o Ollama gerencia tudo internamente.

5.4 Monitorando o espaco em disco

5.4.1 Windows

```
Get-PSDrive -PSProvider FileSystem
```

5.4.2 Linux

```
df -h
```

6 Usando os Modelos

6.1 Uso via linha de comando

A forma mais direta de interagir com um modelo é pelo terminal:

```
# Iniciar uma conversa interativa  
ollama run tinyllama  
  
# Enviar uma pergunta direta (sem modo interativo)  
ollama run qwen2.5-coder:0.5b "Como fazer um loop em Python?"
```

Para encerrar o modo interativo, digite `/bye` ou pressione `Ctrl+D`.

6.2 Uso via Python

6.2.1 Instalacao da biblioteca

```
pip install ollama
```

6.2.2 Exemplo basico

```
import ollama

response = ollama.chat(
    model='qwen2.5-coder:0.5b',
    messages=[
        {'role': 'user', 'content': 'Explique o que e regressao linear.'}
    ]
)

print(response['message']['content'])
```

6.2.3 Chat com historico de conversa

```
import ollama

historico = []

print("Chat iniciado! Digite 'sair' para encerrar.\n")

while True:
    pergunta = input("Voce: ")

    if pergunta.lower() == 'sair':
        print("Encerrando...")
        break

    historico.append({'role': 'user', 'content': pergunta})

    response = ollama.chat(
        model='qwen2.5-coder:0.5b',
        messages=historico
    )

    resposta = response['message']['content']
    historico.append({'role': 'assistant', 'content': resposta})

    print(f"\nModelo: {resposta}\n")
```

6.2.4 Streaming de respostas

O streaming exibe a resposta progressivamente, melhorando a experiencia em hardware mais lento:

```
import ollama

for chunk in ollama.chat(
    model='qwen2.5-coder:0.5b',
    messages=[{'role': 'user', 'content': 'O que e machine learning?'}],
    stream=True
):
    print(chunk['message']['content'], end='', flush=True)
```

6.2.5 Exemplo aplicado a ciencia de dados

```
import ollama

pergunta = """
Em Python, usando pandas e matplotlib, como faco para:
1. Carregar um CSV
2. Calcular estatisticas descritivas
3. Plotar um histograma da coluna idade
"""

for chunk in ollama.chat(
    model='qwen2.5-coder:0.5b',
    messages=[{'role': 'user', 'content': pergunta}],
    stream=True
):
    print(chunk['message']['content'], end='', flush=True)
```

6.3 Uso via API REST

O Ollama expoe uma API REST na porta 11434, compativel com o formato da API da OpenAI:

```
curl http://localhost:11434/api/chat \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen2.5-coder:0.5b",
  "messages": [
    {"role": "user", "content": "O que e uma funcao recursiva?"}
  ],
  "stream": false
}'
```

7 Integrando Modelos nas IDEs com o Plugin Continue

7.1 O que e o Continue?

O **Continue** e um plugin de codigo aberto para IDEs que integra modelos de IA diretamente no ambiente de desenvolvimento. Ele suporta VS Code e toda a familia JetBrains (PyCharm, IntelliJ, etc.) e permite usar modelos locais via Ollama sem custo e sem enviar codigo para servidores externos.

7.2 Instalacao no VS Code

1. Abra o VS Code e pressione **Ctrl+Shift+X**
2. Pesquise por “**Continue**” na barra de busca
3. Clique em **Install**
4. Apos a instalacao, o icone do Continue aparecera na barra lateral esquerda

7.3 Configuracao do arquivo config.yaml

O Continue e configurado pelo arquivo `config.yaml`, localizado em:

- **Windows:** `C:\Users\<seu_usuario>\.continue\config.yaml`
- **Linux/macOS:** `~/.continue/config.yaml`

Para abrir o arquivo diretamente pelo terminal do VS Code:

```
# Windows
code $env:USERPROFILE\.continue\config.yaml

# Linux
code ~/.continue/config.yaml
```

7.3.1 Configuracao com um unico modelo

```
name: Local Config
version: 1.0.0
schema: v1

models:
  - name: Qwen 2.5 Coder 0.5b
    provider: ollama
    model: qwen2.5-coder:0.5b
    apiBase: http://localhost:11434

allowAnonymousTelemetry: false
```

7.3.2 Configuracao com multiplos modelos

```
name: Local Config
version: 1.0.0
schema: v1

models:
  - name: Qwen 2.5 Coder 0.5b
    provider: ollama
    model: qwen2.5-coder:0.5b
    apiBase: http://localhost:11434
```

```

- name: TinyLlama
  provider: ollama
  model: tinyllama:latest
  apiBase: http://localhost:11434

allowAnonymousTelemetry: false

```

Atencao: o campo `tabAutocompleteModel` pode nao ser suportado em versoes mais recentes do Continue (1.2+). Nesse caso, configure o autocomplete diretamente pelo painel da extensao.

7.4 Explicacao dos campos do config.yaml

Campo	Descricao
<code>name</code>	Nome da configuracao – apenas identificacao
<code>version</code>	Versao da sua configuracao
<code>schema</code>	Versao do esquema do Continue – manter v1
<code>models</code>	Lista de modelos disponiveis para o chat
<code>provider</code>	Quem fornece o modelo (ollama para uso local)
<code>model</code>	Nome exato do modelo conforme aparece no ollama list
<code>apiBase</code>	Endereco da API do Ollama (localhost:11434 para uso local)
<code>allowAnonymousTelemetry</code>	Envio de dados anonimos – recomenda-se false

7.5 Principais atalhos do Continue no VS Code

Atalho	Acao
Ctrl+Alt+I	Abre o painel de chat
Ctrl+Shift+L	Envia o codigo selecionado para o chat
Ctrl+I	Edita o codigo selecionado inline com IA
Tab	Aceita sugestao de autocomplete
Esc	Rejeita sugestao de autocomplete

7.6 Referenciando contexto no chat

O Continue permite referenciar partes do projeto diretamente no chat:

Comando	O que referencia
@arquivo.py	Um arquivo especifico do projeto
@codebase	Todo o projeto
@terminal	A saida do terminal integrado
@problems	Os erros e avisos do VS Code

8 Criando um Servidor de IA com Ollama

8.1 Conceito

O Ollama expoe uma API REST local na porta 11434. Por padrao, essa API so aceita conexoes do proprio computador (`localhost`). E possivel configura-lo para aceitar conexoes de outros dispositivos na rede local, transformando seu computador em um servidor de IA compartilhado.

O fluxo de comunicacao funciona da seguinte forma:

- O servidor executa o Ollama com o modelo carregado
- O servidor aguarda requisicoes na porta 11434 de qualquer IP da rede
- Os clientes enviam perguntas para o IP do servidor
- O servidor processa e retorna as respostas

8.2 Configuracao no Linux (Ubuntu)

1. Editar o servico do Ollama

```
sudo systemctl edit ollama.service
```

2. Adicionar a variavel de ambiente

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

3. Reiniciar o servico

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

4. Verificar se esta aceitando conexoes externas

```
# Substitua pelo IP do servidor na sua rede
curl http://192.168.1.100:11434
```

8.3 Configuracao no Windows

No PowerShell como administrador:

```
# Definir variavel de ambiente permanente
[System.Environment]::SetEnvironmentVariable(
    "OLLAMA_HOST",
    "0.0.0.0",
    "Machine"
)

# Reiniciar o servico do Ollama
Stop-Service ollama
Start-Service ollama
```

8.4 Acessando o servidor remotamente

Após configurar o servidor, os outros dispositivos da rede acessam o modelo apontando para o IP do servidor.

8.4.1 Via Python

```
import ollama

# Apontando para o servidor remoto
client = ollama.Client(host='http://192.168.1.100:11434')

response = client.chat(
    model='qwen2.5-coder:0.5b',
    messages=[{'role': 'user', 'content': 'Ola! Voce esta funcionando?'}]
)

print(response['message']['content'])
```

8.4.2 Via Continue no VS Code de outro computador

Aponte o apiBase no config.yaml para o IP do servidor:

```
models:
- name: Qwen no Servidor
  provider: ollama
  model: qwen2.5-coder:0.5b
  apiBase: http://192.168.1.100:11434
```

8.4.3 Via API REST (qualquer dispositivo)

```
curl http://192.168.1.100:11434/api/chat \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen2.5-coder:0.5b",
  "messages": [{"role": "user", "content": "Ola!"}],
  "stream": false
}'
```

8.5 Servidor em dispositivo embarcado: Jetson Nano

A NVIDIA Jetson Nano é uma opção interessante para servidor de IA de baixo consumo energético (5 a 10W), porém o Ollama oficial não suporta sua arquitetura ARM com GPU Maxwell nativamente. A alternativa recomendada é o **llama.cpp**:

```
# Clonar e compilar o llama.cpp com suporte CUDA
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
make LLAMA_CUBLAS=1
```

```
# Iniciar o servidor
./server \
-m models/tinyllama.gguf \
--host 0.0.0.0 \
--port 11434
```

Modelos recomendados para a Jetson Nano (4 GB RAM compartilhada):

Modelo	Viabilidade
tinyllama	Roda bem
qwen2.5-coder:0.5b	Roda bem
llama3.2:1b	Aceitavel
phi3:mini	Lento
llama3.2:3b	Muito pesado

8.6 Boas praticas de seguranca para o servidor

- **Use apenas em rede local confiavel** – nao exponha a porta 11434 para a internet sem autenticacao
- **Configure um firewall** para limitar o acesso apenas aos dispositivos autorizados
- **Monitore o consumo de recursos** – multiplos clientes simultaneos podem sobrecarregar o servidor
- Se precisar de acesso remoto seguro, use uma **VPN** ou **tunel SSH**

9 Referencias e Recursos

- Ollama – Site oficial
- Ollama – Repositorio no GitHub
- Continue – Site oficial
- llama.cpp – Repositorio no GitHub
- Biblioteca Python do Ollama
- Modelos disponiveis no Ollama

Tutorial elaborado com base em experiencia pratica com Ubuntu 22.04 LTS e Windows 11.